EDUGENIE

AI-POWERED LEARNING ASSISTANT

PROJECT REPORT

An educational web application that helps learners ask questions, understand concepts, practice with quizzes, summarize text, and receive structured learning recommendations.

Backend	FastAPI
AI Service	Google Gemini
Frontend	HTML, CSS, JavaScript
Development	Python + VS Code

1. Abstract

EduGenie is an AI-powered learning assistant designed to provide multiple learning activities through a single web interface. The project combines a FastAPI backend with Google Gemini-based generation and a browser frontend. Separate modules handle question answering, concept explanation, quiz generation, text summarization, and learning-path recommendations. The application runs locally through Uvicorn and provides both a browser interface and API endpoints.

2. Introduction

Students may need different forms of support while learning: a direct answer, a simpler explanation, practice questions, a concise summary, or guidance about what to study next. EduGenie brings these functions together in one application.

3. Project Objectives

- Provide an easy-to-use AI learning interface.
- Answer educational questions through a dedicated Q&A function.
- Explain concepts in a learner-friendly manner.
- Generate a structured three-question multiple-choice quiz.
- Summarize supplied educational text.
- Recommend a learning path progressing from beginner to advanced.
- Expose the main capabilities through FastAPI endpoints.
- Keep the application modular for easier maintenance.

4. Technologies Used

Technology	Purpose
Python	Main programming language
FastAPI	Backend web framework and API layer
Uvicorn	Local ASGI application server
Google Gemini	AI content generation
HTML / CSS / JavaScript	Browser frontend
Jinja2	Serving the frontend template
Pydantic	Request/response validation
python-dotenv	Environment configuration
Pytest + HTTPX	API testing

5. System Architecture

The application follows a modular client-server design. The learner interacts with the browser frontend. JavaScript sends the selected task and input to the appropriate FastAPI endpoint. The backend validates the request, calls the relevant feature module, and returns the result to the browser.

Layer	Components
User Interface	templates/index.html, static/style.css, static/app.js
API Layer	main.py and FastAPI routes
Feature Layer	qna.py, explanation_module.py, quiz_module.py, summary_module.py, learning_path.py

AI Layer	gemini_client.py and Google Gemini
Configuration	config.py and .env
Testing / Documentation	tests/ and documentation files

6. Main Modules

Q&A Module

Accepts a learner's question and uses Gemini to generate an educational answer.

Concept Explanation Module

Provides clearer explanations of concepts. The configuration supports Gemini by default and includes an optional local-explainer setting for LaMini-Flan-T5.

Quiz Module

Requests exactly three multiple-choice questions with four options per question and validates the structured response.

Summary Module

Generates a concise summary from supplied text.

Learning Path Module

Generates structured recommendations intended to guide learning from beginner to advanced topics.

7. API Endpoints

Method	Endpoint	Purpose
GET	/health	Check application health
POST	/qa	Answer a learner's question
POST	/explain	Explain a concept
POST	/quiz	Generate a 3-question MCQ quiz
POST	/summarize	Summarize supplied text
POST	/learn/recommendations	Generate learning recommendations

8. Application Output

Insert the final EduGenie output screenshots in this section. The placeholders below are provided so the working interface and generated AI response can be documented in the submitted report.

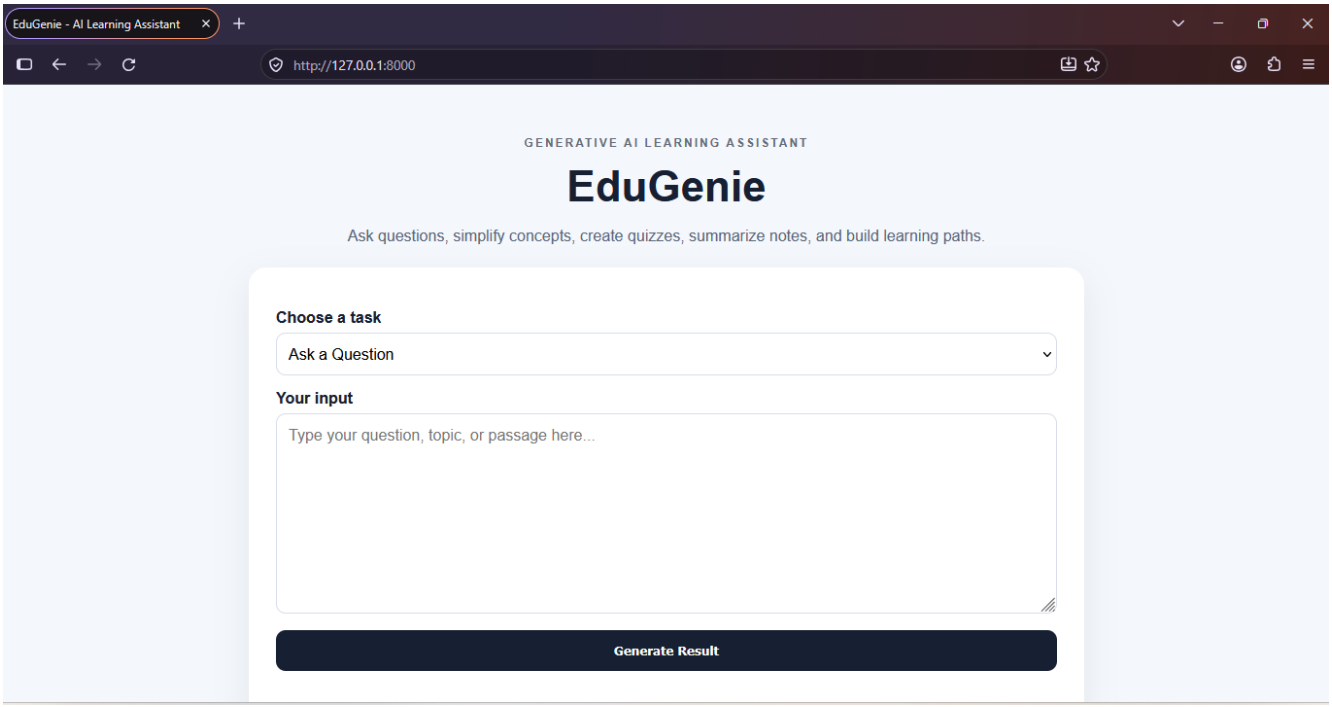


Figure 1 – EduGenie Browser Interface

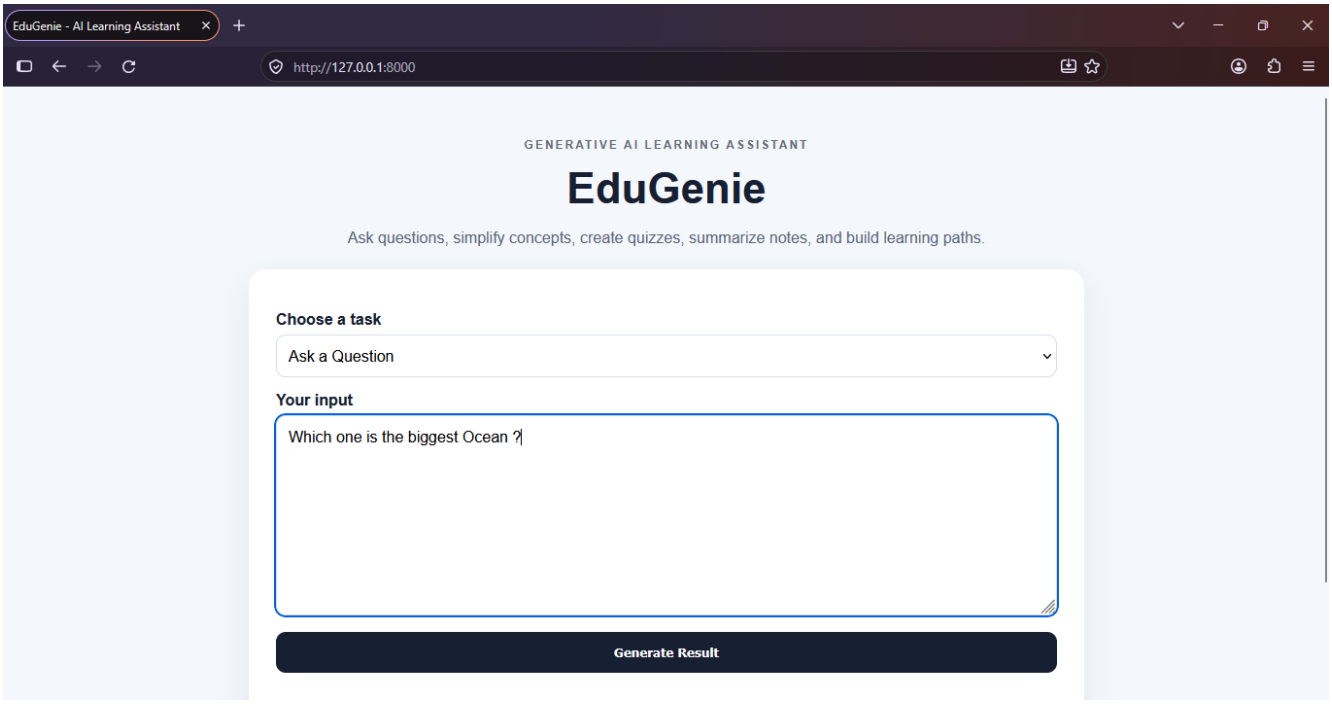


Figure 2 – Input

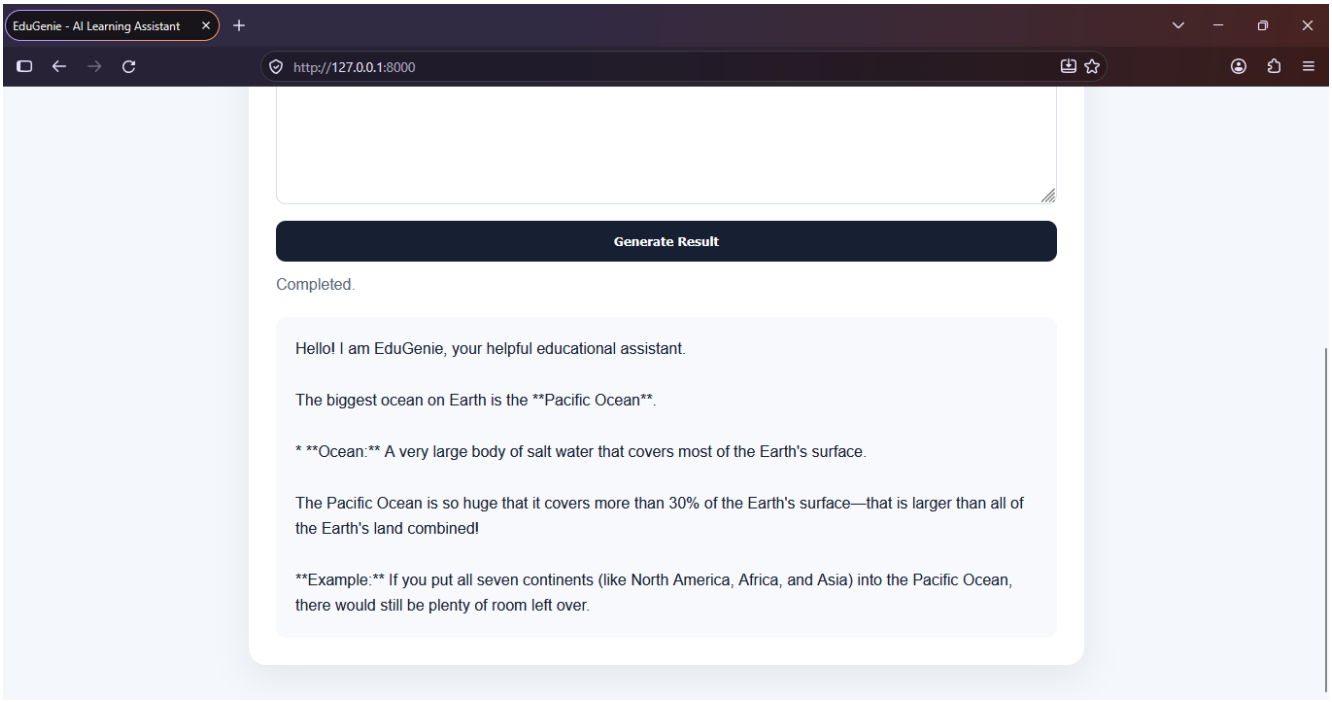


Figure 3 – Output

9. Project Structure

The project is organized into application modules, frontend assets, tests, configuration, and documentation.

```
EduGenie/
├── main.py
├── config.py
├── schemas.py
├── gemini_client.py
├── qna.py
├── explanation_module.py
├── quiz_module.py
├── summary_module.py
├── learning_path.py
├── requirements.txt
├── .env.example
├── .gitignore
├── templates/index.html
├── static/style.css
├── static/app.js
└── tests/
```

10. How to Run the Project

- Open the EduGenie folder in VS Code.
- Activate the existing Python virtual environment (.venv).
- Keep the Gemini API key private inside the .env file.
- Start the server with: python -m uvicorn main:app --reload
- Open http://127.0.0.1:8000 in a browser.
- Use the available learning functions from the EduGenie interface.

11. Security and Configuration

- Configuration is stored through environment variables.
- The Gemini API key should never be placed in screenshots, reports, source control, or shared documents.
- The .gitignore file is included to help prevent private configuration from being committed.
- If an API key is exposed, revoke it and create a replacement key.

12. Testing

The project contains a tests directory with API test configuration. The test setup is designed to validate API behavior while allowing the live AI generation flow to be mocked during testing.

13. Current Development Status

- The FastAPI backend starts successfully under Uvicorn.
- The browser frontend loads from the local application.
- The planned learning modules and API routes are present in the project structure.
- The Python environment and project dependencies have been installed.
- The application is prepared for presentation and output screenshots.

14. Future Enhancements

- Add learner accounts and persistent learning history.
- Store quiz results and learning progress.
- Add difficulty, subject, and topic filters.
- Add richer interactive learning materials.
- Improve error handling and AI-service status messages.
- Deploy the application for access beyond the local computer.
- Improve accessibility and mobile responsiveness.

15. Conclusion

EduGenie combines several AI-assisted educational functions in a single modular web application. Its FastAPI backend separates the learning services while the browser frontend provides a simple entry point for learners. The current implementation provides a foundation that can be extended with personalization, progress tracking, deployment, and richer interactive learning features.

END OF PROJECT REPORT